

Console Input and Encapsulation

Today, we start with reading user input from the console, or command line. We then examine encapsulation, one of the pillars of object-oriented design.

Content Learning Targets

After completing this activity, you should be able to say:

- I can read various data types from the command line using the built-in Scanner class.
- I can explain the difference between the `next()` and `nextLine()` Scanner methods.
- I can use genAI tools to rapidly prototype software designs.
- I can explain encapsulation and evaluate how well given UML designs achieve it.

Process Skill Goals

During the activity, you should make progress toward:

- Identifying similarities and differences between syntax for reading different data types from the command line. (Problem Solving)
- Interpreting UML class diagrams. (Information Processing)
- Understanding the context of a problem/system and applying software design principles. (Problem Solving)



Copyright © 2026 Ian Ludden, based on [prior work](#) of Mayfield et al. This work is under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Part 1 Console Input using Scanner

Java provides the `Scanner` class for reading user input from the console, a.k.a. command line. Review and run *src/consoleInput/ConsoleInputMain.java*. Then, use *ConsoleInputMain.java* and the Java API docs for the `Scanner` class to answer the following questions.

Questions (15 min)

Start time:

1. What type of object does the `Scanner` constructor take as a parameter? In this example, what is the provided argument? To which very familiar Java feature does it connect?
2. Based on the description at the top of the Java API docs for `Scanner`, what are some other options for constructing a `Scanner` object?
3. The example uses the `next()` method to read `String` inputs. Based on the Java API docs, how does `next()` know when to stop?
4. The example uses `nextInt()` and `nextDouble()` to read integer and floating-point inputs. What type of exception do you get if you try entering an invalid `int`/`double` value?
5. Test out the alternate approaches for reading age and height. These use static methods from the `Integer` and `Double` wrapper classes to convert the `String` returned by `scanner.nextLine()` into the appropriate type. What happens if we comment out the `scanner.nextLine();` call before reading the age?
6. (If time allows) Complete the TODOs in *src/consoleInput/ConsolePractice.java*.

Part 2 Prototyping Designs (with genAI)

Sometimes, while we are learning object-oriented design principles, the flaws in a design will not be immediately obvious to us from the UML. In this activity, we see how rapid prototyping with the help of genAI tools can help us make connections between design flaws in UML and how those flaws manifest in Java code.

Demo (15 min)

Recall the Book Tracker Problem from a previous class.

A website tracks books and the kids that read them. For each book the system stores the name and author. For each kid the system stores name and grade level. The teacher enters when a kid reads a particular book. It should be possible to print a report on a book that includes all kids who have read a particular book (with their grade level). It should be possible to print a report on a kid that includes the books (with authors) a particular kid has read.

We saw the following design (Fig. 1) does not function correctly (violating principle 1 a/b), because there is no sane way to look up a book for printing a report or associating with a kid (when handling new reading).

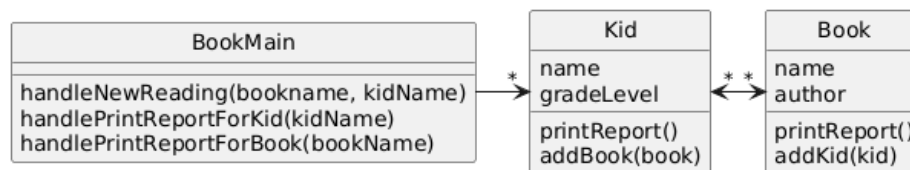


Figure 1: Bad Design A for the Book Tracker Problem ([view PlantUML](#))

7. The src/bookTrackerBadA folder shows the code resulting from Bad Design A. Review the code, and try running it. What issues do you notice?

We saw the following improved design (Fig. 2). To see why this design is better, let's have a genAI tool [refactor](#) our code from Bad Design A to this design.

8. Take notes here on how we prompted the genAI tool.

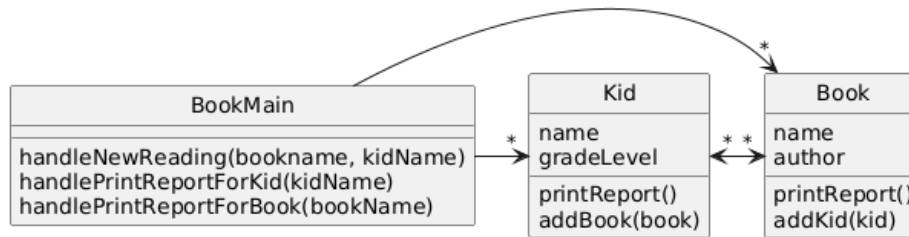


Figure 2: Improved Design for the Book Tracker Problem ([view PlantUML](#))

9. Compare the *BookMain.java* version from Bad Design A with the *BookMain.java* version produced by the genAI tool. What primary differences do you observe?

Part 3 Encapsulation

We continue with one of the pillars of object-oriented software design: *encapsulation*.

Questions (20 min)

Start time:

10. If our software meets all requirements and is bug-free, how could it still be poorly designed?

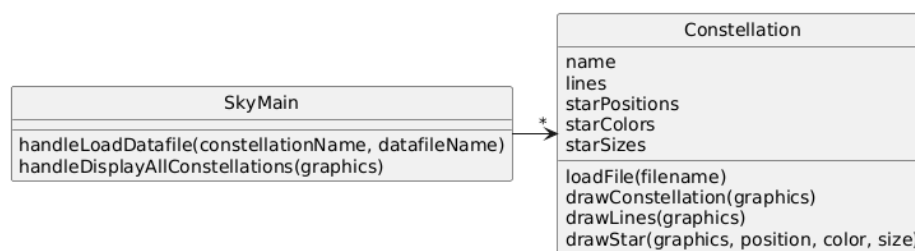
11. Suppose Java's `Math` and `String` classes were merged into one class, putting methods like `sqrt()` and `substring()` next to each other. What would be some negative consequences?

12. In Java, we *could* put all variables and methods in one big class. Why *shouldn't* we?

13. Design Principle 3 (OODP 3) states, “Functionality should be distributed efficiently: (a) No class/part should get too large, and (b) Each class should have a single responsibility.” Restate this principle in your own words, using simpler language.

14. **Constellation Design Problem:** A particular program is designed to load constellations from datafiles and draw them on the screen. The datafiles include the names of the constellations and details about star location, size, and color as well as which stars ought to be connected to draw the constellation. Depending on the star data, each star should be drawn differently (e.g., right size, right color).

Apply our first three object-oriented design principles to evaluate the following design.



([PlantUML source](#))

15. How could we improve this Constellation design?

Definition: Encapsulation

Encapsulation refers to the bundling (in a class) of *related data* with *methods that operate on that data*. A well-designed class forms a sort of “capsule” that has a single job and does it well. We can pass around the capsule (as object references), and other classes can use the methods to get things done without needing to know (or interfering with) the private data inside.

Another benefit of encapsulation is that you can often change *how* a class works on the inside without needing to change any classes that use that class, as long as the public methods stay the same. For example, if someone comes up with a better way of implementing a HashMap, Java can change the HashMap class internals without breaking existing Java code.

Part 4 Design Practice

We end today with some design practice for OODPs 1–3.

Questions (50 min)

Start time:

16. Pizza Restaurant Design Problem: A pizza restaurant needs to calculate the cost of orders and record what pizzas need to be made. An order consists of a number of pizzas, each of which might have toppings, as well as a customer's name and an order date. Each pizza costs \$8 with no toppings. The first 2 toppings cost \$2 apiece. Each additional topping beyond that costs \$1. If a pizza has just peppers, onions, and sausage, that's "The Special" and it costs \$12.

- a) What classes do you think are needed?
- b) Clearly, we need a method to compute a pizza's cost. Where should it go? Why?
- c) Compare [Solution A](#) and [Solution B](#). Which is a better design, and why?
- d) Use a genAI tool to implement both Solution A and Solution B (see Part 2 above). Prompt it to highlight the differences in the two implementations.
- e) Did the genAI tool faithfully implement each design, or did it change some things without telling you? If something changed, what do you think was the motivation? (Feel free to use follow-up prompts.)
- f) We could continue extracting classes (e.g., Customer, Topping). Should we? Explain.

17. (If time allows) Continue with the Car Rental Design Problem (see slides).